CSE 396

Computer Engineering Project

Assignment 3

Module Documentation

Autonomous Fire Detection and Suppression Rover

Group 2

Name	Student ID	Assigned Modules
Cemal GÜMÜŞ	210104004304	MOD-01, MOD-03
Melih KAZANCI	230104004909	MOD-01, MOD-03
Mehmet Alp ATAY	220104004020	MOD-02, MOD-06
Adil Mert ERGÖRÜN	220104004048	MOD-02, MOD-06
Ahmet Halil YAMLI	220104004957	MOD-04, MOD-05
Derya UYSAL	220104004045	MOD-04, MOD-05
Ayşe Feyza SERBEST	220104004052	MOD-07, MOD-08
Görkem UYSAL	230104004174	MOD-07, MOD-08

April 2026

Contents

System Overview **4**

1 MOD-01 — Flame Sensing Hardware **5**
1.1 Module Purpose . . . 5
1.2 Main Responsibilities . . . 5
1.3 Working Principle . . . 5
1.4 Inputs and Outputs . . . 5
1.5 Connection to Other Modules . . . 6
1.6 Technical Design Notes . . . 6
1.7 Known Limitations . . . 6
1.8 Testing and Verification . . . 6

2 MOD-02 — Flame Detection Software **7**
2.1 Module Overview . . . 7
2.2 Constants and Macros . . . 7
2.3 Data Types . . . 8
2.3.1 `flame_status_t` — Return / Error Codes . . . 8
2.3.2 `flame_direction_t` — Detected Flame Direction . . . 8
2.3.3 `flame_data_t` — Aggregated Sensor Reading . . . 8
2.3.4 `flame_cb_t` — Sampling Callback . . . 9
2.4 Public API . . . 9
2.4.1 `flame_init()` . . . 9
2.4.2 `flame_start()` . . . 9
2.4.3 `flame_read()` . . . 9
2.4.4 `flame_get_direction()` . . . 10
2.4.5 `flame_stop()` . . . 10
2.5 Timing and Sampling Behaviour . . . 10
2.6 Inter-Module Protocol: MOD-01 → MOD-02 → MOD-07 . . . 10
2.7 Error Handling Guide . . . 11
2.8 Known Limitations and TODOs . . . 11

3 MOD-03 — Locomotion Hardware **12**
3.1 Module Purpose . . . 12
3.2 Main Responsibilities . . . 12
3.3 Working Principle . . . 12
3.4 Inputs and Outputs . . . 12
3.5 Connection to Other Modules . . . 13
3.6 Technical Design Notes . . . 13
3.7 Known Limitations . . . 13
3.8 Testing and Verification . . . 13

4 MOD-04 — Motion Control Software **14**
4.1 Module Overview . . . 14
4.2 Constants and Macros . . . 14
4.3 Data Types . . . 14
4.3.1 `motion_status_t` — Return / Error Codes . . . 14
4.3.2 `motion_direction_t` — Direction Enumeration . . . 15
4.4 Public API . . . 15
4.5 Known Limitations . . . 15

5 MOD-05 — Distance Sensing Software **16**
5.1 Module Overview . . . 16
5.2 Constants and Macros . . . 16
5.3 Data Types . . . 16
5.3.1 `dist_reading_t` — Sensor Reading . . . 16
5.4 Public API . . . 17
5.5 Inter-Module Protocol: MOD-05 → MOD-07 → MOD-04 . . . 17
5.6 Known Limitations . . . 17

6 MOD-06 — Fire Suppression Hardware **18**
6.1 Module Overview . . . 18
6.2 Main Responsibilities . . . 18
6.3 Constants and Macros . . . 18
6.4 Data Types . . . 19
6.4.1 `suppression_status_t` — Return / Error Codes . . . 19
6.4.2 `suppression_state_t` — Module State Snapshot . . . 19
6.4.3 `suppression_cmd_t` — Command Structure . . . 19
6.5 Public API . . . 20
6.5.1 `suppression_init()` . . . 20
6.5.2 `suppression_activate()` . . . 20
6.5.3 `suppression_deactivate()` . . . 20
6.5.4 `suppression_is_active()` . . . 21
6.5.5 `suppression_get_state()` . . . 21
6.5.6 `suppression_apply_cmd()` . . . 21
6.5.7 `suppression_stop()` . . . 21
6.6 Timing and Hardware Constraints . . . 22
6.7 Inter-Module Protocol: MOD-07 → MOD-06 . . . 22
6.8 Error Handling Guide . . . 23
6.9 Known Limitations and TODOs . . . 23

7 MOD-07 — Autonomy Software **24**
7.1 Module Purpose . . . 24
7.2 Dependencies . . . 24
7.3 FSM Design . . . 24

7.3.1 State Enumeration 24
7.3.2 Transition Diagram 25
7.3.3 State Descriptions 26
7.4 Data Types 26
7.4.1 Input Structure 26
7.4.2 Output Structure 26
7.5 Public API 27
7.6 Operational Flow 27
7.7 Known Limitations 27

8 MOD-08 — Integration and Test Module 29
8.1 Module Purpose 29
8.2 Dependencies 29
8.3 Public API 30
8.3.1 Public Functions 30
8.3.2 Data Types 30
8.4 Module Connections 31
8.5 Operational Flow 32
8.6 Known Limitations and TODOs 32
8.6.1 Current Limitations 32
8.6.2 Planned Improvements 32

System Overview

The Autonomous Fire Detection and Suppression Rover is an embedded system designed to autonomously detect, approach, and extinguish fires. It is built around eight interconnected modules, each with clearly defined responsibilities and interfaces.

Module	Name	Role
MOD-01	Flame Sensing Hardware	Physical IR sensor acquisition
MOD-02	Flame Detection Software	Signal processing and direction estimation
MOD-03	Locomotion Hardware	Chassis, motors, and motor driver
MOD-04	Motion Control Software	High-level motion command translation
MOD-05	Distance Sensing Software	Ultrasonic obstacle detection
MOD-06	Fire Suppression Hardware	Relay-driven water pump control
MOD-07	Autonomy Software	Central FSM and mission decision logic
MOD-08	Integration & Test Module	System-wide validation and simulation

Table 1: System module summary

The overall system structure and inter-module data flow are illustrated in Figure 1.

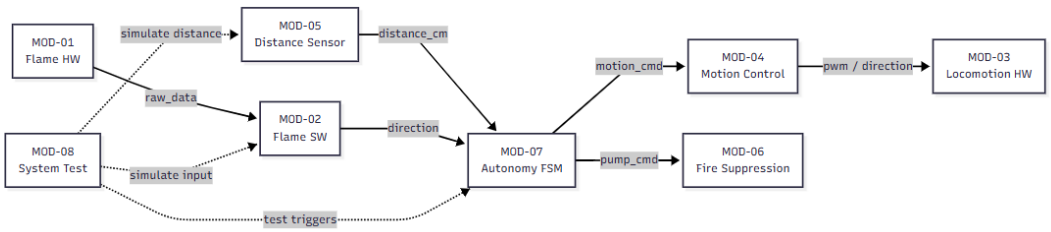

Figure 1: System / module block diagram

1. MOD-01 — Flame Sensing Hardware

1.1. Module Purpose

MOD-01 is responsible for the physical detection of flame-related infrared signals in the environment. Its main task is to acquire raw sensor information and transmit this information to the software layer for further processing.

Within the rover architecture, this module forms the first stage of the fire detection process. It does not make decisions by itself; instead, it acts as the hardware source of flame intensity data.

1.2. Main Responsibilities

- Mounting the flame sensors onto the rover body.
- Wiring the sensors to the microcontroller.
- Ensuring stable analog signal acquisition.
- Reducing noise and ambient light interference.
- Providing raw sensor data for software processing.

1.3. Working Principle

The flame sensors detect infrared radiation emitted by a flame source. Each sensor produces an electrical output corresponding to the detected intensity. These outputs are read by the microcontroller via its ADC channels.

By placing multiple flame sensors in different directions (left, center, and right), the system can compare the measured intensity values and estimate the direction of the flame. MOD-01 therefore provides the physical sensing basis for directional flame localization.

1.4. Inputs and Outputs

Flow	Signal	Description
Input	Infrared energy	IR radiation emitted by flame source
Input	Supply voltage	Power for sensor electronics
Input	Sensor orientation	Physical mounting direction
Output	Raw analog values	10-bit ADC readings (left, center, right)
Output	Digital threshold	Optional digital HIGH/LOW per sensor

Table 2: MOD-01 inputs and outputs

Output Destination: Raw sensor values are sent to **MOD-02 (Flame Detection Software)** for filtering, thresholding, and direction estimation.

1.5. Connection to Other Modules

MOD-01 directly communicates with MOD-02 by supplying raw flame sensor values. MOD-02 then processes this data and forwards flame-related information to the autonomy layer. The processing chain is:

$$\textbf{MOD-01} \rightarrow \textbf{MOD-02} \rightarrow \textbf{MOD-07}$$

This means that MOD-01 indirectly affects rover navigation and suppression decisions, even though it does not control those modules directly.

1.6. Technical Design Notes

- Sensor placement was selected to maximize field of view.
- Wiring was arranged to reduce electrical noise.
- Analog output was preferred because it provides richer intensity information than a digital threshold alone.
- Care was taken to reduce false readings caused by ambient light.
- Sensor orientation and spacing were adjusted to improve directional reliability.

1.7. Known Limitations

- False positives caused by strong ambient light (e.g., sunlight or incandescent lamps).
- Sensitivity to reflective surfaces that scatter IR radiation.
- Unstable readings due to electrical noise on unshielded wiring.
- Incorrect direction estimation if sensor placement or spacing is suboptimal.

1.8. Testing and Verification

- Checking whether each sensor produces a valid output signal.
- Comparing readings for left, center, and right flame positions.
- Testing under different ambient light conditions.
- Verifying successful data transfer to MOD-02.

2. MOD-02 — Flame Detection Software

2.1. Module Overview

MOD-02 is the software layer that reads raw ADC values from MOD-01 (Flame Sensing Hardware) and produces structured, semantically meaningful flame detection results consumed by the Autonomy FSM (MOD-07). It is the only module that knows the *meaning* of IR sensor readings: it applies thresholding, determines which sensor is dominant, and assigns a directional label.

Direction	Module	Interface
Upstream (input)	MOD-01 Flame HW	`flame_hw_read_raw(uint16_t *buffer)` — populates `raw[3]`
Downstream (output)	MOD-07 Autonomy	`flame_data_t` via `flame_read()` or callback

Table 3: MOD-02 inter-module connections

Header file: `Header files/MOD 02/flame_detection.h`

2.2. Constants and Macros

Macro	Value	Rationale
`FLAME_SENSOR_COUNT`	`3`	Three KY-026 IR sensors mounted left, center, right. Mirrors `FLAME_HW_SENSOR_COUNT` from MOD-01.
`FLAME_THRESHOLD`	`500`	ADC reading (0–1023) above which a sensor is considered detecting flame. Mirrors `FLAME_HW_THRESHOLD_DEFAULT` from MOD-01; must be validated experimentally.
`FLAME_SAMPLE_INTERVAL_MS`	`50`	Sampling period in milliseconds (20 Hz). Balances responsiveness vs. CPU load on the ESP32.
`FLAME_ADC_MAX`	`1023`	Full-scale value of the 10-bit ADC on the ESP32. Used for normalization if needed.

Table 4: MOD-02 constants and their rationale

2.3. Data Types

2.3.1. `flame_status_t` — Return / Error Codes

Enumerator	Value	Meaning
`FLAME_OK`	`0`	Operation completed successfully.
`FLAME_ERR_INIT`	`-1`	Hardware initialisation failed. Module must not be used until successful re-init.
`FLAME_ERR_NO_SENSOR`	`-2`	One or more sensors not responding (ADC returns 0 or 1023 persistently).
`FLAME_ERR_TIMEOUT`	`-3`	No new sample produced yet (first call before first sampling cycle).

Table 5: `flame_status_t` enumerator definitions

2.3.2. `flame_direction_t` — Detected Flame Direction

Enumerator	Value	Meaning
`FLAME_DIR_NONE`	`0`	No sensor exceeded threshold; no flame detected.
`FLAME_DIR_LEFT`	`1`	Left sensor (`raw[0]`) has the highest reading above threshold.
`FLAME_DIR_CENTER`	`2`	Center sensor (`raw[1]`) has the highest reading.
`FLAME_DIR_RIGHT`	`3`	Right sensor (`raw[2]`) has the highest reading.

Table 6: `flame_direction_t` enumerator definitions

2.3.3. `flame_data_t` — Aggregated Sensor Reading

This is the primary struct shared with MOD-07. It is populated on every sampling cycle.

Field	Type	Range	Description
`raw[FLAME_SENSOR_COUNT]`	`uint16_t[3]`	0–1023	Raw 10-bit ADC readings. Index 0 = left, 1 = center, 2 = right.
`max_intensity`	`uint16_t`	0–1023	Highest value among `raw[0..2]`. Proxy for flame proximity.
`detected`	`uint8_t`	0 or 1	Non-zero if `max_intensity > FLAME_THRESHOLD`.
`direction`	`flame_direction`	enum	Dominant direction. Only meaningful when `detected == 1`.
`timestamp_ms`	`uint32_t`	ms since boot	Time of reading via `millis()`. MOD-07 should reject data older than 100 ms.

Table 7: `flame_data_t` field descriptions

2.3.4. `flame_cb_t` — Sampling Callback

```
typedef void (*flame_cb_t)(const flame_data_t *data);
```

Listing 1: Callback typedef

Invoked *synchronously* at the end of each 50 ms sampling cycle with a read-only pointer to the freshly populated `flame_data_t`.

2.4. Public API

2.4.1. `flame_init()`

```
flame_status_t flame_init(void);
```

Listing 2: Signature

Attribute	Detail
Pre-condition	None. Safe to call at any point, including after a failed init.
Post-condition	On `FLAME_OK`: ADC pins configured, module ready. On error: state undefined; retry.
Calls	`flame_hw_init()` from MOD-01.
Returns	`FLAME_OK` or `FLAME_ERR_INIT`.
Side effects	Configures ESP32 ADC channels. Does **not** start sampling.

2.4.2. `flame_start()`

```
flame_status_t flame_start(flame_cb_t cb);
```

Listing 3: Signature

Attribute	Detail
Pre-condition	`flame_init()` returned `FLAME_OK`.
Parameters	`cb` – callback per sample; pass `NULL` to use pull model only.
Post-condition	20 Hz sampling begins. Each cycle: reads ADC, computes direction, updates buffer, invokes `cb` if set.
Returns	`FLAME_OK`.
Side effects	Starts repeating timer; consumes CPU proportional to sampling rate.

2.4.3. `flame_read()`

```
flame_status_t flame_read(flame_data_t *out);
```

Listing 4: Signature

Attribute	Detail
Pre-condition	`flame_start()` called. `out` points to valid memory.
Post-condition	`*out` populated with the latest sample.
Returns	`FLAME_OK` if data available; `FLAME_ERR_TIMEOUT` if no sample yet.
Thread safety	Safe to call from main loop while sampling runs.

2.4.4. `flame_get_direction()`

```
flame_direction_t flame_get_direction(void);
```

Listing 5: Signature

Attribute	Detail
Pre-condition	At least one sampling cycle completed.
Returns	Current dominant direction. Returns `FLAME_DIR_NONE` if no sample yet.
Use case	Lightweight FSM polling when only direction is needed.

2.4.5. `flame_stop()`

```
void flame_stop(void);
```

Listing 6: Signature

Attribute	Detail
Pre-condition	None. Safe to call even if not started.
Post-condition	Sampling halted. Last sample retained in buffer.
Side effects	Pending callback cancelled. Call `flame_init()` + `flame_start()` to resume.

2.5. Timing and Sampling Behaviour

Parameter	Value / Notes
Sampling interval	50 ms (20 Hz) — `FLAME_SAMPLE_INTERVAL_MS`
ADC conversion time	$\approx$5 μs per channel; three channels $\approx$15 μs total.
Callback deadline	Must return within $\ll$50 ms to avoid missing the next cycle.
Data staleness limit	MOD-07 should reject `flame_data_t` with `timestamp_ms` older than 100 ms.

Table 8: MOD-02 timing parameters

2.6. Inter-Module Protocol: MOD-01 → MOD-02 → MOD-07

Step 1. Hardware read. MOD-02 calls `flame_hw_read_raw(raw)` (MOD-01). Three 10-bit ADC values written into `raw[0..2]`: left, center, right.

Step 2. Threshold check. For each `raw[i]`, compare to `FLAME_THRESHOLD = 500`.

Step 3. Max intensity. `max_intensity = max(raw[0], raw[1], raw[2])`.

Step 4. Detection flag. `detected = (max_intensity > FLAME_THRESHOLD) ? 1 : 0`.

Step 5. Direction. Index with highest `raw[i]` exceeding threshold → direction enum. If none exceed threshold → `FLAME_DIR_NONE`.

Step 6. Timestamp. `timestamp_ms = millis()`.

Step 7. Publish. Buffer updated. Callback invoked (if registered). MOD-07 calls `flame_read()` or reads callback argument.

2.7. Error Handling Guide

Error Code	Recommended Action
`FLAME_ERR_INIT`	Wait 100 ms, retry `flame_init()`. After 3 consecutive failures, halt and signal MOD-08.
`FLAME_ERR_NO_SENSOR`	Check ADC wiring. Treat as hardware fault; do not continue with faulty sensor.
`FLAME_ERR_TIMEOUT`	Retry after `FLAME_SAMPLE_INTERVAL_MS + 10` ms. If persistent, call `flame_stop()` then `flame_start()`.

Table 9: MOD-02 error handling recommendations

2.8. Known Limitations and TODOs

- **Ambient light interference.** Strong sunlight or incandescent lamps generate IR indistinguishable from flame by the KY-026. Shielded housings are recommended.
- **Single-axis detection only.** Left/center/right only; no elevation angle.
- **Winner-takes-all direction.** A weighted average of all `raw[i]` would provide finer directional precision.
- **Callback thread safety.** If FreeRTOS tasks are used, a mutex must protect the shared buffer.
- **TODO:** Moving-average filter over last N samples before threshold comparison.
- **TODO:** Add `FLAME_ERR_STALE` when `timestamp_ms` has not updated within 2× `FLAME_SAMPLE_INTERVAL_MS`.

3. MOD-03 — Locomotion Hardware

3.1. Module Purpose

MOD-03 is responsible for the physical movement infrastructure of the rover. This includes the motor driver, DC motors, wheel assembly, and the electrical connections required for motion.

This module is the hardware execution layer of the movement subsystem. It receives low-level motion control signals and converts them into real motor actuation.

3.2. Main Responsibilities

- Chassis assembly and structural mounting.
- Motor mounting and alignment.
- Motor driver wiring (L298N connections).
- Battery and power connection setup.
- Enabling directional motor control.
- Enabling PWM-based speed control.

3.3. Working Principle

The locomotion subsystem uses a **differential drive** mechanism. The motor driver receives control inputs from the microcontroller, including direction and PWM-based speed commands. These control signals are translated into motor actuation for the left and right wheels.

By changing the speed and direction of each motor independently, the rover can move forward, backward, turn left, turn right, or stop. MOD-03 therefore provides the physical actuation required for navigation.

3.4. Inputs and Outputs

Flow	Signal	Description
Input	PWM signals	Speed control signals from MOD-04
Input	Direction lines	Motor direction control signals from MOD-04
Input	Battery power	Supply voltage for motors and driver
Output	Wheel rotation	Physical rotation of left and right wheels
Output	Rover movement	Resulting rover displacement in the physical environment

Table 10: MOD-03 inputs and outputs

Input Source: Low-level motion signals are received from **MOD-04 (Motion Control Software)**.

3.5. Connection to Other Modules

MOD-03 receives actuation signals from MOD-04, which converts higher-level motion commands into motor-specific low-level outputs. The control chain is:

MOD-07 → MOD-04 → MOD-03

In this structure: MOD-07 decides what the rover should do, MOD-04 translates this decision into motor commands, and MOD-03 physically executes the movement.

3.6. Technical Design Notes

- Differential drive was selected because it is simple and suitable for indoor rover navigation.
- The L298N driver is used to control two DC motors independently.
- PWM control allows adjustable speed instead of only ON/OFF operation.
- Wheel alignment and chassis stability were considered during assembly.

3.7. Known Limitations

- Wheel slipping during sudden starts without acceleration ramping.
- Imbalance between left and right motors due to manufacturing variation.
- Battery voltage drop reducing motor performance over time.
- Chassis vibration affecting movement stability on rough surfaces.
- Motor driver inefficiency or overheating under sustained load.

3.8. Testing and Verification

- Checking forward and backward motion integrity.
- Checking left and right turning behavior.
- Verifying stop response latency.
- Verifying PWM-based speed adjustment across the 0–255 range.
- Testing motor direction consistency.
- Checking movement stability under battery power at various charge levels.

4. MOD-04 — Motion Control Software

4.1. Module Overview

MOD-04 is the software layer that converts high-level movement requests from MOD-07 into low-level differential drive commands executed by MOD-03 (Locomotion Hardware). It abstracts direction and speed into PWM and GPIO signals.

Direction	Module	Interface
Upstream (input)	MOD-07 Autonomy	`motion_cmd` (direction and speed)
Downstream (output)	MOD-03 Locomotion HW	PWM and direction control lines

Table 11: MOD-04 inter-module connections

4.2. Constants and Macros

Name	Value	Purpose
`MOTION_DEFAULT_SPEED`	`150`	Default PWM speed value (0–255) for normal drive commands.
`MOTION_MIN_THRESHOLD`	`LOCO_HW_MIN_PWM`	Minimum effective PWM level to overcome wheel static friction. Aliases `LOCO_HW_MIN_PWM` (60) from MOD-03.

Table 12: MOD-04 constants

4.3. Data Types

4.3.1. `motion_status_t` — Return / Error Codes

Enumerator	Value	Meaning
`MOTION_OK`	`0`	Operation completed successfully.
`MOTION_ERR_DRIVER`	`-1`	Motor driver communication or actuation failure.
`MOTION_ERR_PARAM`	`-2`	Invalid speed or direction argument was provided.

Table 13: `motion_status_t` enumerator definitions

4.3.2. `motion_direction_t` — Direction Enumeration

Enumerator	Meaning
`MOTION_DIR_STOP`	Stop both motors immediately.
`MOTION_DIR_FORWARD`	Move rover forward.
`MOTION_DIR_BACKWARD`	Move rover backward.
`MOTION_DIR_LEFT`	Perform differential left turn.
`MOTION_DIR_RIGHT`	Perform differential right turn.

Table 14: `motion_direction_t` enumerator definitions

4.4. Public API

Function	Parameters	Return	Description
`motion_init()`	`void`	`motion_status_t`	Initializes motor pins and PWM channels.
`motion_drive()`	`motion_direction_t dir, uint8_t speed`	`motion_status_t`	Converts abstract direction and speed to low-level driver commands.
`motion_set_differential()`	`int16_t left_speed, int16_t right_speed`	`motion_status_t`	Drives left and right motors independently for fine maneuvers.
`motion_get_state()`	`motion_state_t *out_state`	`void`	Copies current direction and speed telemetry into caller buffer.
`motion_emergency_stop()`	`void`	`void`	Immediately sets all motor outputs to stop.

Table 15: MOD-04 public API

4.5. Known Limitations

- Without acceleration ramping and encoder feedback, sudden starts can create wheel slip and heading drift.
- Battery voltage drops can reduce available motor torque and alter response to a fixed PWM value.
- Hardware variability between units may require per-unit PWM calibration.

5. MOD-05 — Distance Sensing Software

5.1. Module Overview

MOD-05 performs ultrasonic distance sensing and reports obstacle proximity to MOD-07. It is tightly coupled with MOD-04 in the rover's close-range navigation loop: MOD-05 continuously feeds distance data to MOD-07, which then commands MOD-04 to drive, turn, or stop based on mission state and safety checks.

Direction	Module	Interface
Upstream (input)	MOD-08 (simulation only)	`simulate_dist` via test harness
Downstream (output)	MOD-07 Autonomy	`dist_reading_t` (distance and timestamp)

Table 16: MOD-05 inter-module connections

5.2. Constants and Macros

Name	Value	Purpose
`DIST_SAFE_THRESHOLD_CM`	`15`	Safe obstacle limit in centimeters. If measured distance is $\leq$ this value, the state is considered unsafe.

Table 17: MOD-05 constants

5.3. Data Types

5.3.1. `dist_reading_t` — Sensor Reading

Field	Type	Description
`distance_cm`	`uint16_t`	Distance computed from ultrasonic echo time.
`timestamp`	`uint32_t`	Reading timestamp in milliseconds via `millis()`.

Table 18: `dist_reading_t` field descriptions

5.4. Public API

Function	Parameters	Return	Description
`distance_init()`	`void`	`int`	Configures trigger and echo pins and prepares sensor I/O.
`distance_read_cm()`	`void`	`uint16_t`	Sends ultrasonic pulse and returns measured distance in centimeters.
`distance_is_unsafe()`	`void`	`int`	Returns `1` if distance $\leq$ `DIST_SAFE_THRESHOLD_CM`, otherwise `0`.

Table 19: MOD-05 public API

5.5. Inter-Module Protocol: MOD-05 → MOD-07 → MOD-04

Step 1. Sensor Acquisition. MOD-05 repeatedly samples range information using `distance_read_cm()`.

Step 2. Data Publishing. Each measurement is propagated as distance context for MOD-07 decision logic.

Step 3. Safety Evaluation. MOD-07 checks unsafe proximity using `distance_is_unsafe()` semantics.

Step 4. Command Generation. MOD-07 computes a motion command (`motion_direction_t` and speed).

Step 5. Actuation. MOD-04 executes the request via `motion_drive()` and emits PWM signals toward MOD-03.

5.6. Known Limitations

- Soft or angled surfaces may absorb or scatter ultrasonic waves, causing inaccurate distance values.
- Timing sensitivity: late processing in MOD-07 may produce stale command execution if distance data is not consumed promptly.
- The module detects only one axis (front-facing); lateral obstacles require additional sensors or rover rotation.

6. MOD-06 — Fire Suppression Hardware

6.1. Module Overview

MOD-06 is the Fire Suppression Hardware module of the Autonomous Fire Detection and Suppression Rover. It is responsible for physically extinguishing a detected fire by operating a relay-driven DC water pump. The module abstracts all low-level hardware interactions (GPIO toggling, relay timing, cooldown enforcement) and exposes a clean, command-driven interface to the Decision and Autonomy module (MOD-07).

MOD-07 (Autonomy SW)	$\xrightarrow{\text{suppression_cmd_t}}$	**MOD-06 (Suppression HW)**

Header file: `Header files/MOD 06/fire_suppression.h`

6.2. Main Responsibilities

- Initialize relay GPIO pin and validate hardware readiness.
- Activate the water pump for a configurable duration.
- Enforce a minimum cooldown period between consecutive activations.
- Deactivate the pump immediately upon receiving a stop command.
- Expose current module state (active flag, timestamps, activation count).
- Execute structured commands issued by MOD-07.
- Safely power down hardware on system shutdown.

6.3. Constants and Macros

Macro Name	Value	Purpose
`SUPPRESSION_PUMP_DURATION_MS`	`3000`	Default pump burst time in milliseconds. Used when command's `duration_ms` field is zero.
`SUPPRESSION_COOLDOWN_MS`	`500`	Minimum interval between consecutive activations. Prevents relay chatter and wear.
`SUPPRESSION_RELAY_PIN`	`7`	GPIO pin number wired to the relay IN signal. Must match the physical board layout.

Table 20: MOD-06 compile-time constants

6.4. Data Types

6.4.1. `suppression_status_t` — Return / Error Codes

Enumerator	Value	Meaning
`SUPPRESSION_OK`	`0`	Operation completed successfully.
`SUPPRESSION_ERR_INIT`	`-1`	Hardware initialisation failed (e.g., GPIO pin not configurable).
`SUPPRESSION_ERR_COOLDOWN`	`-2`	Activation rejected: less than `SUPPRESSION_COOLDOWN_MS` has elapsed since last activation.
`SUPPRESSION_ERR_HARDWARE`	`-4`	Relay or pump hardware fault detected during operation.

Table 21: `suppression_status_t` enumerator definitions

6.4.2. `suppression_state_t` — Module State Snapshot

Field	Type	Units / Range	Description
`is_active`	`uint8_t`	0 or 1	Non-zero while pump relay is closed and water is flowing.
`last_activated_ms`	`uint32_t`	ms since boot	Timestamp of the most recent successful activation. Used to enforce cooldown.
`total_activations`	`uint32_t`	count	Cumulative pump activations since `suppression_init()` was called.

Table 22: Fields of `suppression_state_t`

6.4.3. `suppression_cmd_t` — Command Structure

Field	Type	Valid Range	Description
`activate`	`uint8_t`	0 or 1	Command intent: `1` = activate pump, `0` = deactivate immediately.
`duration_ms`	`uint16_t`	0–65535 ms	Duration override for pump-on time. A value of `0` uses the default `SUPPRESSION_PUMP_DURATION_MS`.
`issued_at_ms`	`uint32_t`	ms since boot	Timestamp set by MOD-07 when command was constructed. Enables stale command detection.

Table 23: Fields of `suppression_cmd_t`

6.5. Public API

6.5.1. suppression_init()

```
suppression_status_t suppression_init(void);
```

Listing 7: Signature

Attribute	Detail
Parameters	None
Returns	`SUPPRESSION_OK` on success; `SUPPRESSION_ERR_INIT` if GPIO cannot be configured.
Pre-conditions	Must be called once before any other MOD-06 function. Typically invoked in `setup()`.
Post-conditions	Relay is open (pump off). `is_active == 0`, `total_activations == 0`, `last_activated_ms == 0`.
Side effects	Sets `SUPPRESSION_RELAY_PIN` to OUTPUT mode and asserts LOW.

6.5.2. suppression_activate()

```
suppression_status_t suppression_activate(uint16_t duration_ms);
```

Listing 8: Signature

Attribute	Detail
Parameters	`duration_ms`: pump-on time; 0 ⇒ use default (3000 ms).
Returns	`SUPPRESSION_OK` on success; `SUPPRESSION_ERR_COOLDOWN` if cooldown not elapsed; `SUPPRESSION_ERR_HARDWARE` if relay fails.
Pre-conditions	`suppression_init()` must have returned `SUPPRESSION_OK`.
Post-conditions	Relay closes; pump starts. `is_active == 1`. `last_activated_ms` and `total_activations` are updated.
Side effects	Drives `SUPPRESSION_RELAY_PIN` HIGH. Schedules auto-deactivation.

6.5.3. suppression_deactivate()

```
suppression_status_t suppression_deactivate(void);
```

Listing 9: Signature

Attribute	Detail
Returns	`SUPPRESSION_OK` on success; `SUPPRESSION_ERR_HARDWARE` if relay fails to open.
Pre-conditions	May be called at any time after `suppression_init()`. Safe to call even if pump is already off.
Post-conditions	Relay opens; pump stops. `is_active == 0`.
Side effects	Drives relay pin LOW; cancels any pending auto-deactivation timer.

6.5.4. `suppression_is_active()`

```
uint8_t suppression_is_active(void);
```

Listing 10: Signature

Returns `1` if the pump relay is currently closed; `0` if idle. Non-blocking; re-entrant and safe to call from any context.

6.5.5. `suppression_get_state()`

```
suppression_status_t suppression_get_state(suppression_state_t *out);
```

Listing 11: Signature

Copies current internal module state into a caller-owned `suppression_state_t` struct for diagnostics, logging, or telemetry. Passing `NULL` is undefined behavior.

6.5.6. `suppression_apply_cmd()`

```
suppression_status_t suppression_apply_cmd(const suppression_cmd_t *
    cmd);
```

Listing 12: Signature

High-level entry point for MOD-07 and MOD-08. Parses a `suppression_cmd_t` and dispatches to `suppression_activate()` or `suppression_deactivate()` accordingly.

6.5.7. `suppression_stop()`

```
void suppression_stop(void);
```

Listing 13: Signature

Graceful shutdown: opens the relay, ensures the pump is off, and sets the GPIO pin to INPUT (high-impedance) to minimize standby current. Safe to call even if the module was never initialized.

6.6. Timing and Hardware Constraints

Parameter	Value / Notes
Cooldown period	500 ms (`SUPPRESSION_COOLDOWN_MS`)
Enforcement method	Checked via `millis() - last_activated_ms` at entry of `suppression_activate()`.
Default burst duration	3000 ms (`SUPPRESSION_PUMP_DURATION_MS`)
Relay GPIO pin	`SUPPRESSION_RELAY_PIN` = 7
Logic level (pump ON)	HIGH (3.3 V on ESP32 GPIO)
Logic level (pump OFF)	LOW (0 V)
Relay switch-on time	≈10 ms (typical mechanical relay)
ESP32 GPIO source/sink	Max 40 mA; relay IN draws ≤5 mA via opto-coupler

Table 24: MOD-06 timing and hardware parameters

6.7. Inter-Module Protocol: MOD-07 → MOD-06

Step 1. FSM enters EXTINGUISH state (MOD-07). MOD-07 determines that the rover has arrived at the target.

Step 2. MOD-07 constructs command struct with `activate = 1`, `duration_ms = 0`, `issued_at_ms = millis()`.

Step 3. MOD-07 calls `suppression_apply_cmd(&cmd)`. MOD-06 checks cooldown, closes relay.

Step 4. Pump runs for 3000 ms. MOD-07 may poll `suppression_is_active()` to track progress.

Step 5. Auto-deactivation. After burst duration, MOD-06 opens the relay. Cooldown timer starts.

Step 6. MOD-07 transitions to VERIFY state. MOD-07 reads flame sensor data (via MOD-02) to confirm extinction.

Step 7. Repeat or return to SEARCH. If flame persists: repeat from Step 2. If extinguished: transition to `SEARCH`.

6.8. Error Handling Guide

Error Code	Trigger Condition	Recommended Action
`SUPPRESSION_ERR_INIT`	GPIO configuration fails during `suppression_init()`	Halt system startup; do not proceed to main loop.
`SUPPRESSION_ERR_COOLDOWN`	`suppression_activa` called within 500 ms of previous activation.	Wait for cooldown; retry after `SUPPRESSION_COOLDOWN_MS`.
`SUPPRESSION_ERR_HARDWARE`	Relay fails to open during `suppression_deact`	Critical fault. Log error; enter safe state; stop all motion.

Table 25: MOD-06 error handling guide

6.9. Known Limitations and TODOs

- No water level sensor; module cannot detect an empty reservoir.
- Auto-deactivation relies on polling in `loop()`; a timing overrun can extend the pump burst.
- `SUPPRESSION_RELAY_PIN` is compile-time; requires recompilation for different boards.
- No interrupt-driven GPIO feedback confirming actual relay state.
- `millis()` wraps at ≈49.7 days; use modular arithmetic to handle wraparound correctly.

7. MOD-07 — Autonomy Software

7.1. Module Purpose

MOD-07 (Autonomy SW) is the central decision-making layer of the rover. It implements a **Finite State Machine (FSM)** that integrates sensor data from multiple modules and issues coordinated commands to all actuator subsystems.

No other module contains mission-level logic. All remaining modules are either hardware drivers or software abstractions over hardware. MOD-07 is the sole component that determines *what the rover should do at any given moment*, making it the architectural apex of the system.

The module's responsibilities are:

- Consuming structured flame detection data from MOD-02 (`flame_data_t`).
- Consuming real-time proximity data from MOD-05 (`distance_cm`).
- Transitioning the FSM across seven defined states based on sensor conditions.
- Issuing motion commands to MOD-04 (`motion_direction_t`, speed).
- Issuing suppression commands to MOD-06 (`suppression_cmd_t`).
- Exposing its current state to MOD-08 for testing and validation.

7.2. Dependencies

Module	Name	Usage
MOD-02	Flame Detection SW	Flame direction and intensity input
MOD-04	Motion Control SW	Receives movement commands
MOD-05	Distance Sensing	Obstacle and range input
MOD-06	Fire Suppression HW	Pump activation output
MOD-08	Test Module	FSM state verification

Table 26: MOD-07 dependencies

7.3. FSM Design

7.3.1. State Enumeration

```
typedef enum {
    STATE_SEARCH,
    STATE_DETECT,
    STATE_APPROACH,
    STATE_AVOID_OBSTACLE,
    STATE_STOP,
    STATE_EXTINGUISH,
    STATE_VERIFY
```

```
} autonomy_state_t;
```

Listing 14: Autonomy FSM state enumeration

7.3.2. Transition Diagram

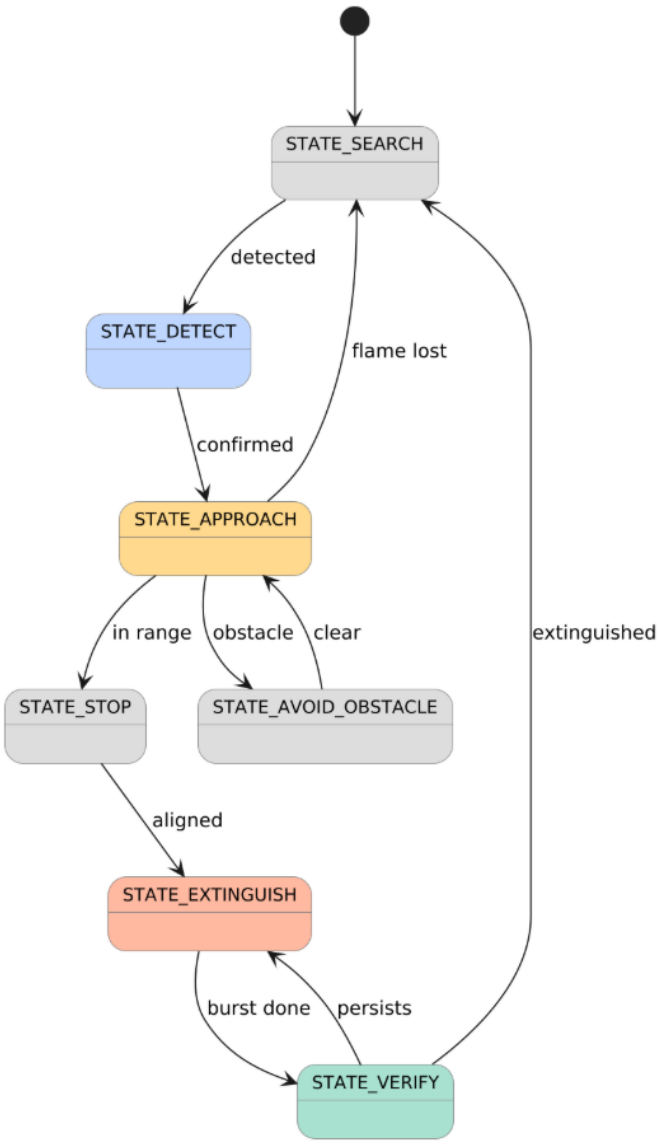

Figure 2: MOD-07 FSM State Transition Diagram

7.3.3. State Descriptions

State	Actuators Active	Description and Exit Condition
`STATE_SEARCH`	MOD-04 (rotate)	Rover rotates in place, polling MOD-02. Exits when `flame_data_t.detected == true`.
`STATE_DETECT`	None	Flame signal confirmed; direction is resolved. Exits immediately to `STATE_APPROACH` once direction is stable.
`STATE_APPROACH`	MOD-04 (drive)	Rover drives toward flame using directional commands. MOD-05 polled every cycle. Exits to `STATE_STOP` when obstacle is unsafe, to `STATE_AVOID_OBSTACLE` on lateral obstacle, or to `STATE_SEARCH` if flame is lost.
`STATE_AVOID_OBSTACLE`	MOD-04 (steer)	Rover executes avoidance maneuver. Exits back to `STATE_APPROACH` when path is clear.
`STATE_STOP`	None	All motors halted; rover aligns with flame. Exits to `STATE_EXTINGUISH`.
`STATE_EXTINGUISH`	MOD-06 (pump on)	Water pump activated via `suppression_cmd_t`. MOD-02 polled for extinction. Exits to `STATE_VERIFY` after pump burst.
`STATE_VERIFY`	None	Confirms `flame_data_t.detected == false`. Exits to `STATE_SEARCH` if flame persists; exits to `STATE_STOP` (idle) if extinguished.

Table 27: FSM state descriptions and exit conditions

7.4. Data Types

7.4.1. Input Structure

```c
typedef struct {
    flame_data_t flame_data;
    uint16_t     distance_cm;
} autonomy_input_t;
```

Listing 15: autonomy_input_t definition

Field	Type	Description
`flame_data`	`flame_data_t`	Flame direction and intensity data from MOD-02
`distance_cm`	`uint16_t`	Obstacle distance in centimeters from MOD-05

Table 28: Input structure fields

7.4.2. Output Structure

```c
typedef struct {
    motion_direction_t motion_cmd;
    uint8_t            speed;
    uint8_t            pump_on;
} autonomy_output_t;
```

Listing 16: autonomy_output_t definition

Field	Type	Description
`motion_cmd`	`motion_direction_t`	Direction command issued to MOD-04
`speed`	`uint8_t`	PWM speed value (0–255)
`pump_on`	`uint8_t`	Pump activation flag sent to MOD-06

Table 29: Output structure fields

7.5. Public API

Function	Description
`autonomy_init()`	Initializes the FSM to `STATE_SEARCH`.
`autonomy_update(in)`	Updates FSM with the latest sensor input struct.
`autonomy_get_output(out)`	Returns the current command output struct.
`autonomy_reset()`	Resets FSM to initial state.
`autonomy_get_current_state()`	Returns the active `autonomy_state_t` for telemetry or MOD-08 validation.

Table 30: MOD-07 public API

7.6. Operational Flow

1. Search for flame by rotating in place.
2. Detect and confirm flame target; resolve direction.
3. Move toward flame using directional drive commands.
4. Avoid obstacles if encountered during approach.
5. Stop at safe distance from flame.
6. Activate water pump and begin suppression.
7. Verify flame extinction via MOD-02.
8. Return to search mode if flame is still detected.

7.7. Known Limitations

- No advanced path planning; direct obstacle avoidance is reactive only.

- No timeout watchdog for any state; a stuck state loops indefinitely.
- No speed ramping; abrupt starts cause wheel slip (see MOD-04 limitations).
- Sensor noise from MOD-02 or MOD-05 may cause spurious state transitions.

8. MOD-08 — Integration and Test Module

8.1. Module Purpose

MOD-08 serves as the system-wide validation and quality assurance layer for the Autonomous Fire Detection and Suppression Rover. Its primary responsibility is to verify that all eight modules operate correctly both individually and in concert before and during field deployment.

The module provides two complementary testing modes: a **real hardware self-check** that exercises every physical peripheral on the rover, and a **simulation mode** that injects synthetic sensor data into the system, allowing full Finite State Machine (FSM) validation without requiring a physical fire source or any hardware.

8.2. Dependencies

Module Name	ID	Usage in MOD-08
Flame Sensing HW	`MOD-01`	Sensor presence validation during self-check
Flame Detection SW	`MOD-02`	Full detection pipeline verification via `flame_data_t`
Locomotion HW	`MOD-03`	Motor driver response test
Motion Control SW	`MOD-04`	Movement command verification via `motion_direction_t`
Distance Sensing	`MOD-05`	Ultrasonic obstacle detection validation
Fire Suppression HW	`MOD-06`	Relay and pump activation test
Autonomy SW	`MOD-07`	End-to-end FSM behavior testing with injected triggers

Table 31: MOD-08 dependencies

8.3. Public API

8.3.1. Public Functions

Function	Parameters	Return	Description
`test_init()`	`void`	`void`	Initialises internal state and links to all dependent modules.
`test_set_mode()`	`test_mode_t mode`	`void`	Switches between `TEST_MODE_REAL` and `TEST_MODE_SIMULATION`.
`test_run_self_check()`	`void`	`test_status_t`	Sequentially validates all hardware peripherals; returns first failure code or `TEST_OK`.
`test_get_report()`	`test_report_t *report`	`void`	Fills caller-owned struct with per-subsystem pass/fail flags.
`test_set_simulation_data()`	`const test_simulation_input_t *data`	`void`	Injects synthetic flame and distance data for simulation runs.
`test_update()`	`void`	`void`	Periodic monitoring tick; logs anomalies during continuous operation.

Table 32: MOD-08 public functions

8.3.2. Data Types

Type	Kind	Fields / Values
`test_status_t`	enum	`TEST_OK`, `TEST_FAIL_SENSOR`, `TEST_FAIL_DISTANCE`, `TEST_FAIL_MOTOR`, `TEST_FAIL_PUMP`
`test_mode_t`	enum	`TEST_MODE_REAL`, `TEST_MODE_SIMULATION`
`test_report_t`	struct	`sensor_ok`, `distance_ok`, `motor_ok`, `pump_ok` (all `int`)
`test_simulation_input_t`	struct	`flame_data` (`flame_data_t` from MOD-02), `distance_cm` (`uint16_t`)

Table 33: MOD-08 data types

8.4. Module Connections

MOD-08 occupies a unique position in the architecture: it has **read access to every module's output** for validation purposes, and **write/inject access** to MOD-02, MOD-05, and MOD-07 for simulation. In production mode it is passive; in test mode it becomes the data source.

From	To	Signal / Data	Direction	Notes
MOD-08	MOD-02	simulate_flame	08 → 02	Injected flame_data_t for simulation runs
MOD-08	MOD-05	simulate_distance	08 → 05	Injected distance_cm for simulation runs
MOD-08	MOD-07	test_triggers	08 → 07	FSM state injection for behavioral testing
MOD-01	MOD-08	sensor_status	01 → 08	Sensor presence confirmed during self-check
MOD-03	MOD-08	motor_status	03 → 08	Motor driver response during self-check
MOD-05	MOD-08	dist_reading	05 → 08	Ultrasonic ping result for validation
MOD-06	MOD-08	pump_status	06 → 08	Relay activation confirmation
MOD-07	MOD-08	fsm_state	07 → 08	Current FSM state read for behavioral validation

Table 34: MOD-08 inter-module signal map

8.5. Operational Flow

#	Real Mode	Simulation Mode
1	`test_init()` — link all modules	`test_init()` — link all modules
2	`test_set_mode(TEST_MODE_REAL)`	`test_set_mode(TEST_MODE_SIMULATION)`
3	`test_run_self_check()` — ping each peripheral	`test_set_simulation_data()` — load scenario
4	`test_get_report()` — read pass/fail flags	`test_update()` — inject data into MOD-02/05/07
5	`test_update()` — continuous monitoring loop	Read `autonomy_get_current_state()` to assert

Table 35: MOD-08 operational flows

8.6. Known Limitations and TODOs

8.6.1. Current Limitations

- Test routines depend on controlled environment conditions; field noise may produce false failures.
- Simulation scenarios are limited to a single static input snapshot; dynamic sequences are not yet supported.
- No detailed per-component error logging; `test_report_t` provides only binary pass/fail per subsystem.
- `test_update()` does not yet implement continuous flame-loss or motor-stall detection.

8.6.2. Planned Improvements

Add granular error codes to `test_report_t` (e.g., which sensor index failed).

Extend simulation to support multi-step scenarios with changing distance and flame direction over time.

Implement automated test logging to serial output or SD card for field debugging.

Integrate continuous system monitoring with alert thresholds in `test_update()`.

Add a dedicated stress-test mode that cycles `STATE_EXTINGUISH` repeatedly to validate relay cooldown (MOD-06).